

Typpe|Racer

Guide de démarrage — contributeurs

1. Vue d'ensemble

TypeRacerDiscord est un jeu de vitesse de frappe embarqué dans Discord comme Activity (Embedded App SDK) — backend Rust, frontend React/TypeScript.

Deux familles de Run : **Practice** (solo, entrée libre façon Monkeytype) et **Race** (compétitif, en Room). Une Race se joue selon un **Mode de jeu** — la règle qui décide comment elle se gagne :

- **Normal** — premier à taper le texte entier, exactement.
- **Floor is lava** — à intervalle fixe, le joueur le moins avancé est brûlé ; le dernier survivant gagne, sans ligne d'arrivée.
- **Spam** — un mot répété en boucle sur un texte infini ; la victoire va au seuil de répétitions ou au plafond de temps, selon lequel des deux tombe en premier.

Le salon Discord affiche l'activité en cours (Rich Presence) : statut mis à jour côté client au fil des transitions de Run/Race (`updateActivity`, `frontend/src/discord.ts`), visible par les autres membres du salon vocal sans qu'ils aient besoin d'ouvrir l'Activity.

Le vocabulaire précis du domaine (Run, Mode, Mode de jeu, Réglage de salon, Difficulté, Gap, Brûlé, Devancé, PB...) est défini dans `CONTEXT.md`, à la racine du dépôt — ce document n'en reprend aucune définition, il y renvoie systématiquement.

2. Stack technique

Backend — Rust, `axum` (HTTP + WebSocket), `tokio`, `sqlx` (SQLite, migrations), `reqwest` (proxy de citations), sert aussi le build statique du frontend en production.

Frontend — TypeScript, Vite, `@discord/embedded-app-sdk` (intégration Activity/OAuth Discord), `chart.js` (graphiques de résultats). Aucun framework de vue — DOM manipulé directement.

Tests — `vitest` côté frontend (domaine TypeScript, référence de l'algorithme de score), `cargo test` côté backend (parité avec le calcul Rust + le store SQLite).

3. Démarrage local

Prérequis : Rust (`cargo`) et Node (`npm`). Aucune clé Discord n'est requise pour développer : le backend démarre en **mode dev**, où le Bearer token sert directement de `player_id` (jouable au navigateur, hors Discord).

```
# Backend (port 8080)
cd backend
cargo run

# Frontend (port 5173, proxy /api /token /ws → 8080) — 2e terminal
cd frontend
npm install
npm run dev

# Tests
cd frontend && npx vitest run
cd backend && cargo test
```

La mise en place complète pour tester **dans** Discord (tunnel cloudflared, URL Mappings du portail développeur, pièges CSP connus) est déjà documentée dans `README.md` — non reprise ici pour ne pas la faire diverger.

4. Décisions d'architecture

Les choix de conception qui ne se lisent pas depuis le code seul sont enregistrés en ADR, `Docs/adr/`. Un résumé d'une phrase chacun pour naviguer sans ouvrir les dix-sept fichiers :

1. **Persister le texte cible verbatim** — le Replay stocke le texte tel que tapé plutôt que de le régénérer depuis un seed.
2. **L'identité d'affichage n'est jamais persistée** — nom et avatar sont annoncés à la connexion et oubliés, jamais écrits côté serveur.
3. **Les Quotes ne produisent jamais de PB** — leur longueur variable rend le Config bucket incomparable d'une Quote à l'autre.
4. **Solo démarre sans décompte** — $t = 0$ devient la première frappe au lieu de masquer le texte pendant 3 s.
5. **Trigram Drill, un Mode séparé de Drill** — il retient aussi le caractère qui suit une faute, pas seulement celui qui la précède.
6. **Le cursus Apprendre passe à 100 leçons** — extension du cursus existant plutôt qu'un second système parallèle.
7. **Le décompte de Race passe à 7 s** — un réglage produit, choisi comme tel, pas une unité de mesure à figer.
8. **Une Room s'identifie par une clé** — salon vocal Discord OU code de partie, selon comment on y entre.
9. **Une Race n'a pas de Mode** — elle a une Source de texte (Quote ou Mots), un axe distinct du Mode solo.
10. **RaceOver porte les résultats** — pas seulement l'ordre d'arrivée : le message porte directement le podium.
11. **Play of the Game** — un duel choisi par le serveur, rejoué au ralenti sur une horloge partagée.
12. **Le Leaderboard fait confiance au recompute** — aucun anti-triche dédié : le recompute autoritaire du backend suffit déjà.
13. **Difficulté et l'état Failed** — Normal/Expert/Master, ce dernier seul disponible et autoritaire en Race.
14. **Plein écran : le contenu s'échelonne** — jamais de scroll, le contenu se met à l'échelle de la fenêtre.
15. **Floor is lava** — un Mode de jeu à part entière, sans ligne d'arrivée : on gagne en survivant.
16. **Spam** — texte infini, victoire au seuil de répétitions ou au plafond de temps.
17. **Seam Mode de jeu** — table déclarée pour `game_mode`, et un contrat de retour explicite pour les Réglages de salon.